

ACME Employee Salary Management Implementation Approach (Simple Guide)

Rails API + React SPA | Flow diagrams + logic walkthrough
github.com/rohitmuk857007-cyber/salary_management-Rohit

This short PDF explains **what we built**, **how data flows**, and the **salary history logic** in plain language — with diagrams you can follow without reading the whole codebase.

1. Goal in one line

Give ACME's HR Manager a web app to manage ~10,000 employees' salaries across countries, replace Excel, and answer "how does the org pay people?" with simple insights.

2. High-level system flow

HR uses the React UI. The UI calls the Rails API. The API reads/writes SQLite (local) or Postgres (production).

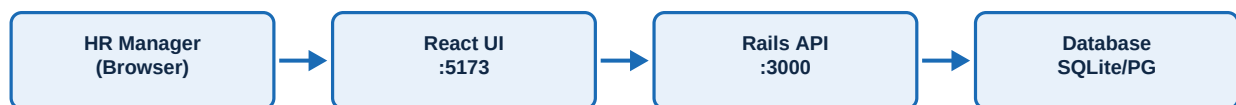


Figure 1 — Request path from HR to database

3. User journey (HR flow)

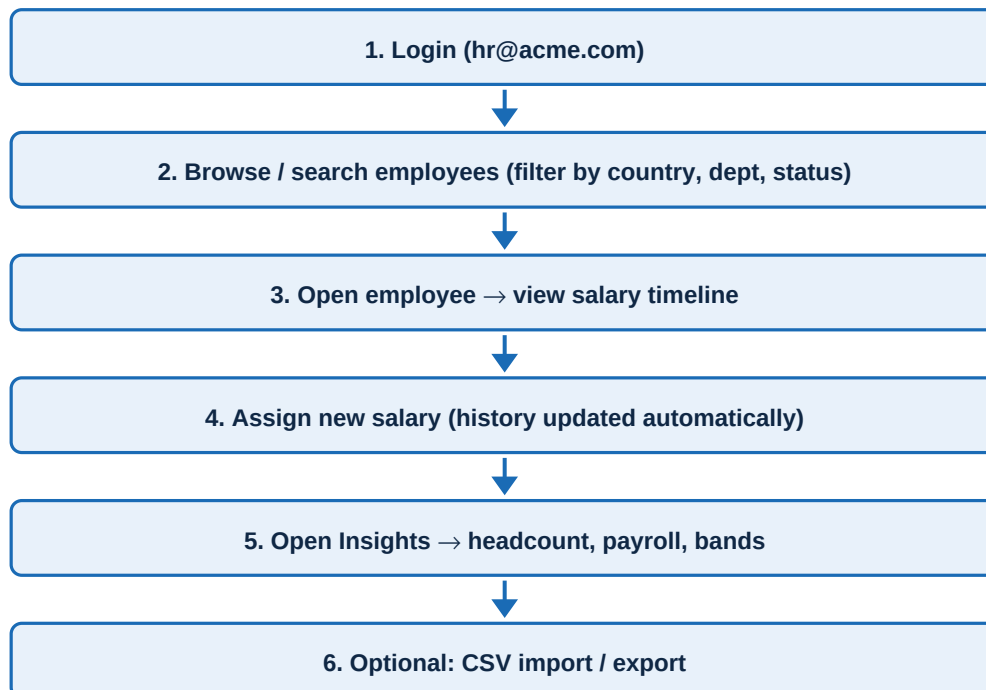


Figure 2 — Day-to-day HR workflow in the app

4. Implementation approach (order of work)

We built bottom-up: clarify scope → data model → API → seed/tests → UI → docs. Each step was an incremental git commit so the story is reviewable.

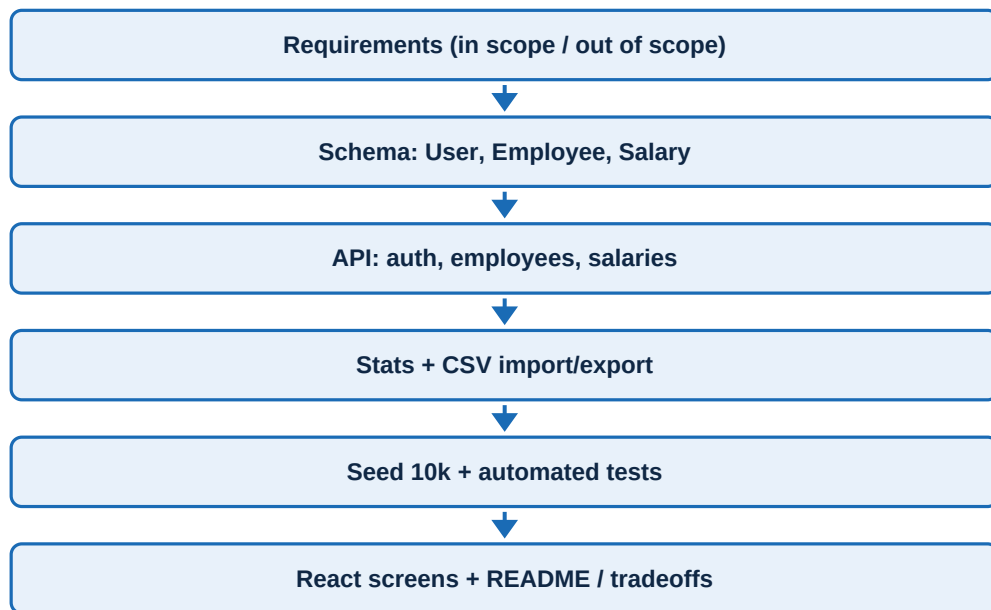


Figure 3 — Build sequence

5. Data model (simple)

- **User** — HR login (email + password).
- **Employee** — code, name, email, country, department, hire date, status.
- **Salary** — amount in **cents** (integer), currency (USD/INR/...), effective_from, effective_to (nullable), reason. Belongs to one employee.

Current salary = the row where effective_to is empty (NULL).

6. Core logic: salary history (never overwrite)

When HR sets a new salary, we do **not** edit the old amount. We close the old open row and insert a new one. That keeps a full timeline (raises, corrections) and makes audits easy.

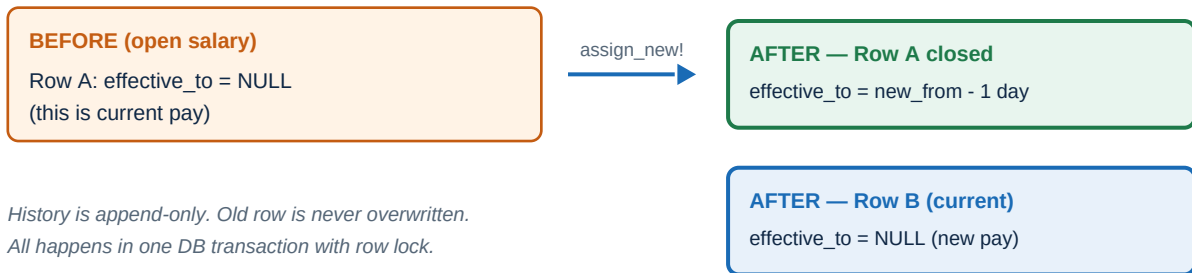


Figure 4 — What happens inside Salary.assign_new!

Steps inside the transaction

1. Lock any open salary rows for that employee.
2. Set their effective_to to the day before the new salary starts.
3. Insert the new row with effective_to = NULL (now current).
4. Commit — either all succeeds or nothing changes.

Code entry point: backend/app/models/salary.rb → Salary.assign_new!

7. API map (what the UI calls)

Area	Endpoints
Auth	POST /api/v1/auth/login, DELETE logout, GET me
Employees	CRUD + filters: q, country, department, status, page
Salaries	GET/POST /api/v1/employees/:id/salaries
Stats	GET /api/v1/stats/overview
CSV	POST imports/employees, GET exports/employees.csv

8. Insights logic

Stats use **active employees** + **current salaries** only. Totals are grouped **by currency** (we do not fake a single global number across INR and USD). Bands bucket amounts into under 50k / 50–100k / 100–150k / 150k+.

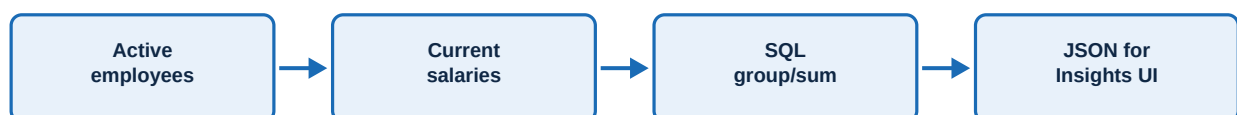


Figure 5 — Insights calculation path

9. Seeding 10,000 employees

Batch insert (`insert_all`, 1000 at a time). Mix of countries/currencies/departments. Each employee gets 1–3 salary history rows. Override size with `SEED_COUNT=100` for a quick smoke test. Demo login: `hr@acme.com` / `password123`.

10. Main design choices

- **SQLite locally** for easy demo; Postgres for production.
- **Session cookies** (not JWT) for simple login/logout with the SPA.
- **Salary history** instead of overwrite — audit-friendly.
- **Out of scope**: tax engines, SSO, multi-role RBAC, live FX conversion.

Repo: https://github.com/rohitmuk857007-cyber/salary_management-Rohit